arXiv:2511.12723v1 [cs.LG] 16 Nov 2025

LAYA: Layer-wise Attention Aggregation for Interpretable Depth-Aware Neural Networks

A Preprint

Gennaro Vessio
Department of Computer Science
University of Bari Aldo Moro
Bari, Italy
gennaro.vessio@uniba.it

Abstract

Deep neural networks typically rely on the representation produced by their final hidden layer to make predictions, implicitly assuming that this single vector fully captures the semantics encoded across all preceding transformations. However, intermediate layers contain rich and complementary information—ranging from low-level patterns to high-level abstractions—that is often discarded when the decision head depends solely on the last representation. This paper revisits the role of the output layer and introduces **LAYA** (*Layer-wise Attention Aggregator*), a novel output head that dynamically aggregates internal representations through attention. Instead of projecting only the deepest embedding, LAYA learns input-conditioned attention weights over layer-wise features, yielding an interpretable and architecture-agnostic mechanism for synthesizing predictions. Experiments on vision and language benchmarks show that LAYA consistently matches or improves the performance of standard output heads, with relative gains of up to about one percentage point in accuracy, while providing explicit layer-attribution scores that reveal how different abstraction levels contribute to each decision. Crucially, these interpretability signals emerge directly from the model's computation, without any external post hoc explanations. The code to reproduce LAYA is publicly available at: https://github.com/gvessio/LAYA.

Keywords Deep learning · Attention mechanisms · Neural architecture design · Representation learning · Interpretability

1 Introduction

Deep learning architectures have achieved outstanding success across computer vision, natural language processing, and multimodal tasks. Despite differences in modality and structure, most deep networks share a common principle: the final prediction is obtained from the representation learned by the *last hidden layer*. Formally, given an input x processed by a model with parameters θ, the forward pass produces a sequence of L hidden representations

$$h_1, h_2, \ldots, h_L = f_\theta(x),$$

where f_θ denotes the composition of the hidden layers. The final prediction is then obtained by applying a task-specific output head to the last representation:

$$\hat{y} = \phi\big(W h_L + b\big),$$

where $\phi(\cdot)$ denotes a task-specific activation function. This conventional design implicitly assumes that the last embedding h_L encapsulates all relevant semantic information extracted by the network.

However, empirical and theoretical studies suggest that different layers capture distinct and complementary aspects of the input: early layers encode local patterns, middle layers capture structural relations, and deeper layers represent abstract semantics (e.g., [Bau et al., 2017, Tenney et al., 2019]). Popular architectures such as ResNet [He et al., 2016] and DenseNet [Huang et al., 2017] partially address this issue through skip or dense connections, facilitating gradient flow and feature reuse. Nevertheless, these mechanisms operate within the feature extractor, while the output head remains a static projection over the final representation. Similarly, multi-scale fusion models such as FPN [Lin et al.,

2017] and Transformer-based architectures [Vaswani et al., 2017, Dosovitskiy, 2020] exploit hierarchical information during feature learning but still collapse the entire representational hierarchy before classification.

This work revisits the output stage of deep neural networks. The central idea is that the output layer should not depend exclusively on the deepest representation but instead integrate information across depth. To this end, **LAYA** (*Layer-wise Attention Aggregator*) is introduced as a mechanism that learns to weight and combine all hidden representations $\{h_i\}_{i=1}^{L}$ in an input-dependent manner.

Each hidden state is mapped into a shared latent space through a lightweight adapter $g_i(\cdot)$, ensuring comparability across layers. LAYA then replaces the standard classifier head with an attention-based aggregator:

$$h_{\text{agg}} = \sum_{i=1}^{L} \alpha_i(x)\, g_i(h_i),$$

where $\alpha_i(x)$ are learnable, input-conditioned coefficients reflecting the relevance of each layer for the current sample. The final prediction is computed as:

$$\hat{y} = \phi(W h_{\text{agg}} + b).$$

This formulation transforms the output layer from a passive projection into an active module that reasons over the network's internal states. Crucially, the attention weights $\alpha_i(x)$ provide an explicit and quantitative attribution signal, revealing which layers influence the prediction. This yields interpretability that emerges directly from the model's computation, without relying on external post hoc techniques.

In summary, this work makes three main contributions. First, a core limitation of standard neural architectures is examined: the final prediction relies exclusively on the last hidden representation, disregarding the structured hierarchy of intermediate features learned throughout the network. Second, LAYA is introduced as a lightweight, architecture-agnostic output module that applies input-conditioned attention across all hidden layers, enabling depth-aware aggregation while leaving the backbone and its computational pipeline unchanged. Third, a systematic experimental framework is presented across both vision and language tasks, evaluating LAYA against competitive baselines and assessing the interpretability afforded by its layer-wise attention scores.

The rest of the paper is structured as follows. Section 2 surveys related work. Section 3 presents the LAYA mechanism and its integration into standard architectures. Section 4 describes the experimental setup, benchmarking results, and interpretability analyses. Section 5 discusses broader implications and potential directions for future work.

2 Related Work

Hierarchical representation learning. Deep neural networks organize computation hierarchically, with early layers capturing low-level features and deeper ones encoding progressively abstract semantics. Early convolutional models such as LeNet [LeCun et al., 2002], AlexNet [Krizhevsky et al., 2012], and VGG [Simonyan and Zisserman, 2014] demonstrated the effectiveness of deep hierarchies but suffered from vanishing gradients and feature degradation. Residual Networks (ResNet) [He et al., 2016] introduced skip connections to facilitate gradient flow and feature reuse, enabling the training of substantially deeper architectures. Densely Connected Networks (DenseNet) [Huang et al., 2017] further extended this principle by concatenating outputs from all preceding layers to encourage feature reuse and reduce redundancy. While these designs improve representational richness and allow information to flow across layers, the final classifier still receives a *single, collapsed representation* h_L. Although h_L is a function of all preceding transformations, the output layer cannot access intermediate features explicitly nor modulate their contribution based on the input; all depth-specific information is irreversibly compressed into the last embedding.

Multi-scale and feature fusion models. Many architectures explicitly integrate features across multiple spatial scales or abstraction levels. Feature Pyramid Networks (FPN) [Lin et al., 2017] and BiFPN [Tan et al., 2020] merge hierarchical features to enhance object detection and segmentation, while HRNet [Sun et al., 2019], U-Net++ [Zhou et al., 2018], and DeepLab [Chen et al., 2018] leverage encoder–decoder symmetries to preserve both spatial resolution and semantic depth. Vision Transformers (ViT) [Dosovitskiy, 2020] and hierarchical variants such as Swin Transformer [Liu et al., 2021] similarly exploit multi-stage attention. CrossViT [Chen et al., 2021] extends this direction by allowing cross-attention between representations extracted at different patch granularities. Cross-layer attention designs [Huang et al., 2022] further highlight that intermediate representations carry complementary task-relevant signals. More recently, LayerFusion [Zheng et al., 2024] demonstrated that aggregating intermediate representations can yield more contextual embeddings, though this fusion typically occurs *within* the encoder. In all these cases, hierarchical information is successfully incorporated during feature learning; however, the classifier still collapses the resulting hierarchy into a single representation before prediction, with no mechanism for input-dependent weighting across layers.

Output-level reasoning and interpretability. Despite substantial progress in feature fusion, the output layer in most architectures remains a static linear projection. Early-exit models such as BranchyNet [Teerapittayanon et al., 2016] attach auxiliary classifiers to intermediate layers for efficiency, and deep supervision approaches [Lee et al., 2015] encourage multiple layers to contribute training signals. However, these methods do not aggregate features at inference time. Ensemble strategies [Lakshminarayanan et al., 2017] combine multiple predictors, yet they require multiple forward passes. ScalarMix [Peters et al., 2018], used in contextual embeddings such as ELMo, mixes representations from different layers, but the mixing weights are global parameters shared across all inputs, limiting adaptability. Attention mechanisms such as Bahdanau attention [Bahdanau et al., 2014] learn relevance scores dynamically, but are typically applied across sequence or token dimensions rather than across network depth.

Positioning of LAYA. LAYA differs from prior approaches by introducing *output-level depth reasoning*: it operates strictly at the prediction stage and assigns *input-conditioned* attention weights to representations from all layers. Instead of relying on a single terminal embedding, LAYA aggregates $\{h_i\}_{i=1}^{L}$ through learned coefficients $\alpha_i(x)$, making the final decision explicitly dependent on multiple abstraction levels. This yields intrinsic interpretability, as the attention weights function as layer-attribution scores indicating which depths contribute most to each prediction. LAYA is architecture-agnostic, adds minimal parameter overhead, and requires no modification to the backbone or its training pipeline. Compared to ScalarMix-style approaches, which learn a *global*, input-independent mixture of layer representations, LAYA produces *per-sample* relevance weights and therefore adapts the depth contribution dynamically to each input. In contrast to cross-layer fusion mechanisms integrated within the encoder—such as LayerFusion or cross-layer attention—LAYA does not modify the backbone or alter feature learning: it acts as a *pure output head* that reasons over already-computed internal states. This design makes LAYA easy to retrofit onto existing architectures, even frozen ones.

3 Method

Motivation. Deep networks learn hierarchical representations, with different layers capturing increasingly abstract information. However, the standard output head relies exclusively on the final embedding h_L, discarding useful signals computed at earlier depths. LAYA (*Layer-wise Attention Aggregator*) addresses this limitation by allowing the prediction stage to integrate information from all layers in an input-dependent manner.

Overview. LAYA redefines the standard output head as a depth-aware aggregation module. Instead of relying solely on h_L, LAYA assigns input-dependent attention weights to all hidden representations $\{h_i\}_{i=1}^{L}$ and forms a weighted combination prior to prediction. This enables the model to adaptively emphasize different abstraction levels based on the input.

Formulation. Consider a backbone network with L hidden layers:

$$h_i = f_i(h_{i-1}), \quad i = 1, \ldots, L, \qquad h_0 = x.$$

In standard architectures, the prediction is computed as:

$$\hat{y} = \phi(W h_L + b).$$

LAYA generalizes this by treating all $\{h_i\}$ as relevant signals. Each h_i is first projected into a common latent space of dimension d^* by a learnable adapter:

$$g_i : \mathbb{R}^{d_i} \to \mathbb{R}^{d^*}, \qquad z_i = g_i(h_i).$$

Then, an input-conditioned aggregation is computed as:

$$h_{\text{agg}} = \sum_{i=1}^{L} \alpha_i(x)\, z_i,$$

where the coefficients $\alpha_i(x)$ are attention weights specific to each input sample. The final prediction is obtained as:

$$\hat{y} = \phi(W h_{\text{agg}} + b).$$

Attention mechanism. Unlike standard attention mechanisms that operate over spatial or temporal dimensions, LAYA performs attention over the depth dimension of the network. The attention coefficients $\alpha_i(x)$ are generated by a small meta-network operating on the enriched representations of all layers. Each z_i can be optionally transformed by a shared mapping:

$$u_i = \psi(z_i),$$